

Designer Performance

How the module works: data sources, scoring, permissions

KPI Dashboard | System documentation

This document describes the Designer Performance module of the KPI Dashboard: what it measures, where every piece of data comes from, how the two scores are calculated, and who is allowed to see and change each thing.

What the module measures

It scores how well a project's paperwork was handled, in two stages. The score belongs to the designer assigned to the project, and everything is keyed by SO number.

PHASE 1 - Project intake

A Go / No-Go checklist of required documents.

Score = 100 minus delay in delivering them.

A manual review then closes the phase as Complete, Deficient or Deferred.

PHASE 2 - Project closure

Red flags raised during production, taken from designer-tagged project notes.

Score = 100 minus a penalty per day each flag stays open.

ONE NUMBER PER DESIGNER

The global KPI is the average of both phase scores across the designer's projects. A project that never reached Phase 2 contributes only its Phase 1 score.

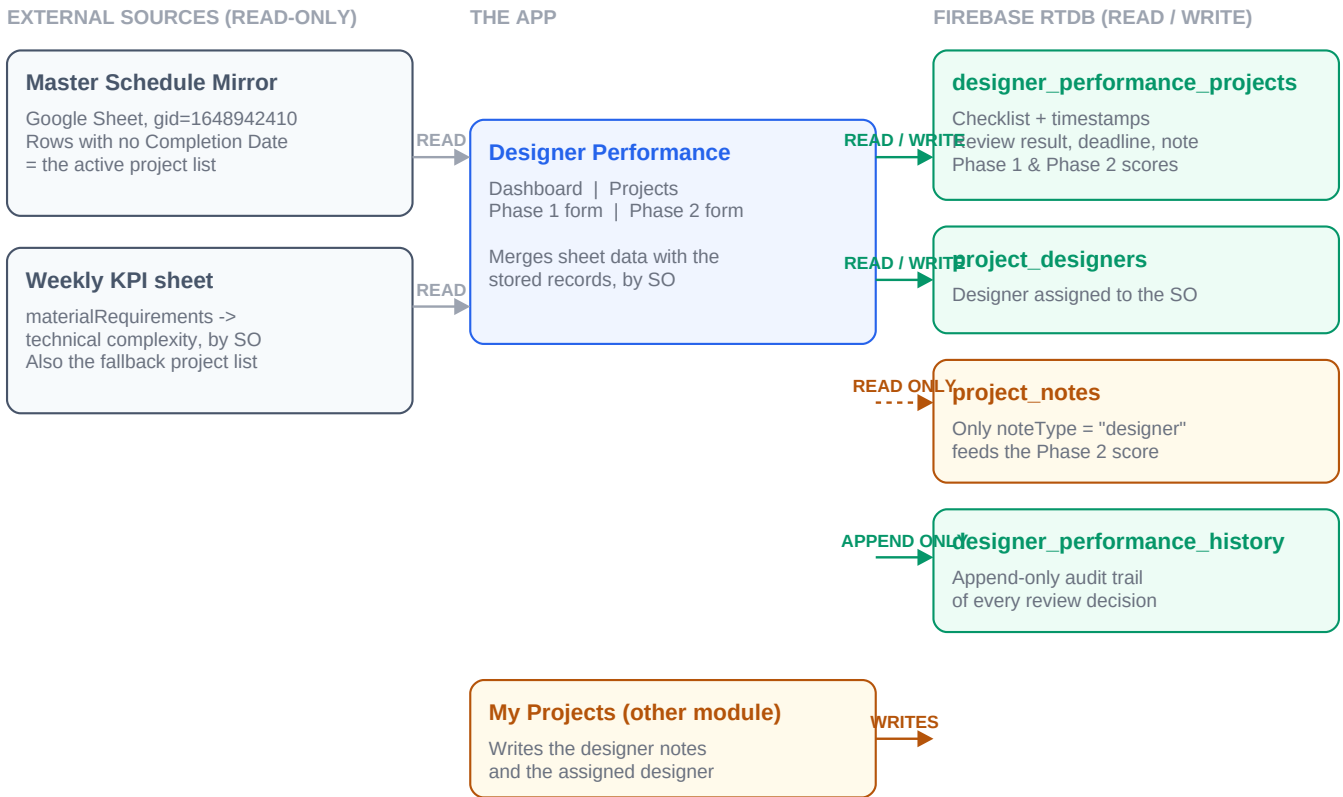
Reading this document

- Section 1 - where every field comes from, and what is read-only.
- Section 2 - who can view and edit what, and which rules are actually enforced.
- Section 3 - the Phase 1 score, in full.
- Section 4 - the manual review result and the note the designer receives.
- Section 5 - the Phase 2 score.
- Section 6 - known limitations, including what is missing for an audit.

SECTION 1

Where the data comes from

There is no single source. Each field has its own, and that is the most important thing to understand about this module. Google Sheets are read-only: nothing from Designer Performance is ever written back to a spreadsheet.



TWO MODULES WRITE THE SAME DATA

project_notes and project_designers are written by My Projects, not by Designer Performance. Designer Performance only reads the notes. For the assigned designer, it re-reads the value immediately before saving and refuses to overwrite it if it changed - the only concurrency guard in the module.

SECTION 2

Who can see and change what

Roles assignable from inside the app: engineer, engineer_nester, administrative and designer. A hidden super-admin role (engineer-admin) exists and can only be set from the Firebase Console.

Permission matrix

ACTION	WHO CAN DO IT	ENFORCED WHERE
Open the Designer Perf. tab	Everyone, including designers	UI
See Dashboard and Projects	Everyone, including designers	UI
See the Phase 1 / Phase 2 forms	Everyone except designers	UI
Write the intake record	Everyone except designers	Database rule
Choose the review result	Everyone except designers	Database rule
Approve with missing paperwork	administrative only	Database rule
Create / resolve designer notes	engineer, engineer_nester, engineer-admin	Database rule
Generate the automatic note	Everyone except designers	API role check
Write the review history	Everyone except designers; entries can never be edited or deleted	Database rule

ONE RESIDUAL GAP

Approving with missing paperwork is enforced when a project first moves into Approved. A project that is already Approved can be edited by any non-designer, so the checklist could in principle be emptied afterwards in a second step. That escape exists on purpose: without it, an engineer could not fix a typo on a project the administrative team had already force-approved.

There is no such thing as an assigned reviewer

Any approved non-designer user can mark any project Complete, Deficient or Deferred. There is no list of authorised reviewers, no reviewer assigned per project, and no requirement that it be the engineer in charge of that job.

What a designer sees

Designers can open the module and the project list, but not the intake forms. They receive a shareable link to a single project, which opens its record after logging in: review result, the written note, the missing documents and the deadline. The link is not public - the record carries the client name and internal notes.

SECTION 3

The Phase 1 score

Every project starts at 100. Points are lost for each day a required document is late, counted in business days from the date the project was registered.

Checklist items

KCD file | JL Contract | Quote (complete by room) | Quote breakdown | Credit Card Form | Drawings signed by client | Final Measurements (only if it applies)

Penalty rates, per item

ITEM	FIRST 4 BUSINESS DAYS	FROM DAY 5	MAXIMUM
All standard documents	-1 pt / day	-2 pts / day	-20 pts
Final Measurements	-0.1 pt / day	-0.4 pt / day	-20 pts

WHY FINAL MEASUREMENTS IS DIFFERENT

It depends on scheduling, not only on the designer, so it penalises an order of magnitude softer than the rest. Weekends never count: a document requested on Friday and delivered on Monday is one day late, not three.

Two rules that prevent unfair scores

- Same-day delivery costs nothing. Ticking every box on the day the project was registered leaves the score at 100, however old the project is.
- A checklist item added to the product later does not penalise projects retroactively - its clock starts on the day the item shipped.
- The recorded date of each tick can be corrected by hand, for paperwork that arrived earlier than it was logged.

Missed review deadlines

A Deficient or Deferred result carries a deadline. Choosing the result costs nothing by itself - it is a diagnosis, not a fault. What costs points is not fixing it in time: -1 pt per business day past the deadline, -2 from day 5, capped at -20.

THIS PENALTY IS DERIVED, NEVER STORED

It is recalculated every time the project is read, so an overdue project keeps losing points without anyone reopening the form. Curing it late does not erase what was accrued: the project moves to Complete but keeps its deadline, stamped with the date it was resolved.

The review result

The checklist drives the score. What approves the phase is a manual quality review, recorded as one of three results taken from the engineering status definitions.

RESULT	MEANING	REQUIRES	PHASE 2
Complete	All documentation present and the manual review passed. Ready for engineers to begin work.	Full checklist	Enabled
Deficient	Reached engineering, but errors, wrong measurements or inconsistencies were found. Returned to the designer.	Written notice + cure deadline	Blocked
Deferred	Sold, but on hold because vital files (contract, .job KCD file) are missing.	Written reason + deadline	Blocked

Deficient and Deferred do not require a complete checklist - recording that something is missing is exactly what they are for. Only Complete demands the full set.

The note the designer receives

When a result is chosen, the note is drafted automatically in English, listing the checklist items that are still missing. The engineer can edit it freely before saving; it is the same field, so the automatic text is a starting point, not a replacement for a human comment.

How the draft is produced

1. The application builds the text from the checklist - the facts never come from the model.
2. Gemini rewrites it into prose, through a server-side endpoint that keeps the API key private.
3. If the model is unavailable, times out, or drops one of the documents from the list, the deterministic text is used instead. The feature can never block a save.

IT NEVER OVERWRITES WHAT YOU TYPED

The draft is only regenerated while the note is untouched - including if you start typing while the model is still writing. The "Draft again" button is the only thing that overwrites, because it is an explicit request.

The Phase 2 score

Phase 2 measures friction during production. It starts at 100 and subtracts a penalty for every red flag raised against the project - that is, every project note tagged "designer". Notes of any other type are ignored.

Penalty per note, per day open

SEVERITY	FIRST 4 DAYS	FROM DAY 5	MAXIMUM PER NOTE
Green	-0.5 pt / day	-1 pt / day	-10 pts
Yellow	-1 pt / day	-2 pts / day	-20 pts
Red	-2 pts / day	-4 pts / day	-40 pts

The caps are all equivalent to twelve business days open, so the severity ranking holds at every point: a red flag always weighs four times a green one.

When the clock stops

- Resolving a note freezes its penalty at the day it was resolved. It is not refunded - the friction happened.
- Notes created before the feature shipped start counting from the release date, so nobody carries age they had no way to clear.
- Closing the project in Phase 2 freezes the whole breakdown, which is stored with the project.

BUSINESS DAYS, SAME AS PHASE 1

Days open are counted in business days. A note raised on Friday and cleared on Monday counts as one day, not three - nobody works Saturday, so that is not anyone's delay. Both phases now use the same calendar.

Audit trail

Every change to the review result is recorded. The result itself is stamped with the person who set it, and each change is appended to a separate history node instead of overwriting the previous one, so the full sequence survives.

What a history entry holds

When it happened, who did it, the resulting status and review result, the written note, the deadline, and the Phase 1 score at that moment.

AN AUDITOR ASKS...	CAN THE MODULE ANSWER IT?
Why was this project deferred?	Yes - the written reason is mandatory
When was it deferred?	Yes - stored as a timestamp
Who deferred it?	Yes - the result is stamped with its author
Was the result changed before?	Yes - the full sequence is kept
Who raised this red flag?	Yes - notes store their author
Who ticked this checklist item?	No - only the date is recorded

THE TRAIL CANNOT BE REWRITTEN

History entries can be created but never modified or deleted - that restriction is enforced by the database itself, not by the application. Someone who wanted to hide a change would have to change the rules in the Firebase Console, which is a separate and visible act.

What is still missing

- The checklist records when each document was ticked, but not by whom. The tick date can also be corrected by hand, and that correction leaves no trace.
- There is no concept of an assigned reviewer: any approved non-designer can review any project.
- Concurrency is unguarded except for the assigned designer. Two people saving the same project at once - the last write wins, though both attempts appear in the history.

SCOPE OF THIS DOCUMENT

Everything described here reflects the code as of the date on the cover. The database rules referenced are those in the repository; the authoritative boundary is whatever is published in the Firebase Console, which must be confirmed separately - the rules on this page only take effect once published.